HLD Level 2.7

Design a Distributed Rate Limiter — Complete Interview Question & Answer

> **Interview question: Design a rate limiter for a large API platform. It must enforce limits per user/API key/IP and optionally per endpoint, support burst control, work across many application servers, add very low latency, and remain reliable when the shared rate-limit store is slow or unavailable.**

1. Requirements

Functional: decide ALLOW/DENY for each request, support configurable policies such as 100 requests/minute, limit by user/API key/IP/endpoint, support bursts where appropriate, return retry metadata, and allow policy updates.

Non-functional: very low decision latency, horizontal scalability, high availability, bounded memory, atomic updates, fairness, resistance to hot keys and predictable behavior during dependency failure.

2. Where the Limiter Lives

```
Client -> Load Balancer / API Gateway -> Rate Limiter -> Application
```

Placing rate limiting near the edge prevents rejected traffic from consuming expensive application/database resources.

3. Capacity Example

```
Peak API traffic = 1,000,000 requests/sec
Each request requires a rate-limit decision
Decision budget should usually be sub-millisecond to a few milliseconds
```

At this scale, a remote database round trip for every request can become a major bottleneck. The design must minimize state access cost and distribute load.

4. Rate-Limit Key

```
key = user_id
or api_key
or source_ip
or user_id + endpoint
```

The key determines fairness. A global limit protects the entire service; per-user limits protect tenants from one another; endpoint-specific limits protect expensive operations.

5. Fixed Window Counter

```
Rule: 100 requests/minute

key = user123:12:30
counter++
if counter <= 100 -> ALLOW
else -> DENY
```

Advantages: simple, memory efficient, fast. Weakness: boundary bursts. A client can send 100 requests at 12:30:59 and another 100 at 12:31:00, effectively allowing 200 requests in about one second.

6. Sliding Log

Store timestamps of recent requests for each key and remove timestamps outside the window. It is accurate but memory/operation cost can be high for heavy users.

```
timestamps = [t1, t2, t3, ...]
remove timestamps < now-window
if count < limit -> append now and allow
```

7. Sliding Window Counter

Approximate a true sliding window using current and previous fixed-window counts weighted by elapsed time. It reduces boundary bursts while using much less memory than a full timestamp log.

8. Token Bucket

```
Bucket capacity = 100 tokens
Refill = 10 tokens/sec
Request cost = 1 token

if token available:
consume token -> ALLOW
else:
DENY
```

Token bucket supports controlled bursts: idle time accumulates tokens up to capacity, allowing a short burst while enforcing the long-term refill rate.

> **Token bucket is often the strongest default interview choice when the requirement includes both sustained rate control and reasonable bursts.**

9. Leaky Bucket

Requests enter a bounded bucket/queue and leave at a steady rate. It smooths output traffic but can add queueing delay. Token bucket controls admission; leaky bucket emphasizes shaping.

10. Choosing an Algorithm

Fixed window: simplest, cheap, boundary burst issue.
Sliding log: accurate, expensive at high volume.
Sliding counter: practical approximation.
Token bucket: excellent for sustained rate + bursts.
Leaky bucket: useful when smooth downstream rate is the main goal.

11. Single-Process Token Bucket State

```
struct Bucket {
double tokens;
steady_clock::time_point last_refill;
};
```

On each request, calculate elapsed time, add elapsed*refill_rate up to capacity, then consume the request cost if enough tokens remain.

12. Why Monotonic Time?

For elapsed-duration calculations in C++, use a monotonic clock such as std::chrono::steady_clock. Wall-clock adjustments should not create or remove tokens unexpectedly.

13. Thread Safety

If multiple application threads update the same bucket concurrently, token calculation and decrement must be atomic as one logical operation. A simple local implementation can shard keys across mutexes/maps rather than using one global lock.

14. Sharded Local Limiter

```
hash(key) -> shard

Shard 0: mutex + buckets
Shard 1: mutex + buckets
...
Shard N: mutex + buckets
```

Sharding reduces contention because unrelated users do not serialize behind one mutex.

15. The Distributed Problem

Suppose there are 10 API servers and user123 is allowed 100 requests/minute. If each server independently allows 100, the real fleet-wide limit becomes approximately 1000.

```
Client -> LB -> App1 local counter
-> App2 local counter
-> AppN local counter
```

A globally enforced limit needs coordinated/shared state or a deliberate approximation strategy.

16. Shared Rate-Limit Store

```
Clients
|
API Gateway Fleet
|
Rate-Limit Client
|
Distributed In-Memory Store
|
ALLOW / DENY
```

A fast distributed key-value/in-memory system can hold counters/token state. Updates must be atomic so concurrent gateways cannot overspend the same bucket.

17. Atomicity

Do not implement token checking as separate remote GET and SET operations because two gateways can both read the same value and both allow. Use a server-side atomic primitive/script/transaction that refills, checks and consumes in one operation.

```
atomic operation:
read state
compute refill
if enough tokens -> decrement + ALLOW
else -> DENY
persist new state
```

18. Partitioning

```
partition = hash(rate_limit_key) % N
```

Partition by rate-limit key so state scales horizontally. Consistent hashing or store-native sharding can reduce disruption when nodes change.

19. Hot Keys

A celebrity account, shared NAT IP or global API key may receive enormous traffic and concentrate requests on one shard. Options include hierarchical limits, local token leasing, key redesign, dedicated partitions or intentionally approximate distributed counters.

20. Local Cache / Token Leasing

To avoid one remote store operation per request, the global limiter can lease a small block of tokens to each gateway.

```
Global bucket has 1000 tokens
Gateway A leases 50
Gateway B leases 50
Gateway C leases 50
```

Gateways consume leased tokens locally and request more when needed. This greatly reduces remote operations but can temporarily overshoot or waste unused leases after crashes.

21. Accuracy vs Availability Trade-Off

Strict global limits require tighter coordination and can increase latency/dependency risk. Approximate/local limits improve availability and performance. The correct choice depends on whether the limiter protects cost, fairness, security, or merely smooths load.

22. Hierarchical Rate Limits

```
Global service limit
|
Tenant limit
|
User/API-key limit
|
Endpoint limit
```

A request may need to satisfy multiple buckets. Expensive endpoints can consume more tokens than cheap operations.

23. Weighted Requests

```
GET /status -> cost 1 token
POST /heavy-report -> cost 10 tokens
```

Token bucket naturally supports weighted cost, which is useful when requests have very different backend resource impact.

24. Response to Rejected Client

Return an appropriate status (commonly HTTP 429 for HTTP APIs) and, when useful, metadata indicating retry timing or remaining quota. Do not make clients guess blindly.

25. Configuration

Store policies separately from hot counters. Examples: tenant tier, endpoint, capacity, refill rate, burst size and effective time. Gateways can cache policy configuration with versioning/TTL.

26. Policy Update

When a policy changes, propagate/version it carefully. Decide whether existing token state is preserved, clamped to the new capacity, or reset. Avoid inconsistent interpretations across gateways.

27. Memory Management

Millions of inactive users should not leave bucket objects forever. Expire state after a TTL based on how long it would take the bucket to refill fully or after a safe inactivity interval.

28. Failure: Rate-Limit Store Unavailable

The interviewer will often ask: fail-open or fail-closed?

Fail-open: allow traffic when limiter dependency fails. Better availability, weaker protection.
Fail-closed: reject when uncertain. Strong protection, risks outage.

Choose per endpoint. Login/security-sensitive or expensive operations may prefer fail-closed; low-risk user traffic may use a local emergency limit/fail-open strategy.

29. Degraded Mode

A useful compromise is a local fallback limiter with conservative quotas while the distributed store is unavailable. This keeps some protection without making the shared limiter a total availability dependency.

30. Store Latency

If remote rate-limit latency rises, application latency rises too. Monitor it separately and enforce tight client-side timeouts. Never let requests wait indefinitely for the limiter.

31. Clock Issues

If the distributed algorithm relies on timestamps, prefer server-side/store time or carefully defined monotonic/local elapsed time. Unsynchronized wall clocks across gateways can create inconsistent token refill calculations.

32. Retry Storms

Rejected clients may all retry at the same time. Return retry guidance and encourage exponential backoff with jitter. Otherwise the limiter can create synchronized waves of traffic.

33. Protecting the Limiter Itself

The limiter is on the hot path and can become a target/bottleneck. Use connection pooling, bounded client queues, request timeouts, sharding, replicas/failover where appropriate and strict resource limits.

34. Observability

Track allowed/sec, denied/sec, denial ratio, decision latency P50/P95/P99, distributed-store latency/errors, hot keys, tokens leased, local fallback usage, policy-cache hit rate, shard CPU/memory and rejected traffic by tenant/endpoint.

35. Abuse vs Capacity Limiting

Rate limiting is not a complete security system. IP limits can punish users behind NAT and attackers can rotate addresses. Authentication, quotas, anomaly detection and WAF/security controls may complement it.

36. C++ Local Token Bucket

```
lock shard
now = steady_clock::now()
elapsed = now - last_refill
tokens = min(capacity, tokens + elapsed * rate)
if tokens >= cost:
tokens -= cost
allow = true
else:
allow = false
unlock
```

Keep the critical section short. Do not perform network calls or application work while holding the bucket/shard lock.

37. Lock-Free?

A lock-free implementation is not automatically better. Token refill modifies multiple related values, making correctness subtle. Start with sharded locking, measure contention, and optimize only if necessary.

38. Complete Architecture

```
CLIENTS
|
v
Load Balancer / Gateway
|
+---------+---------+
v v
Policy Cache Rate-Limit Client
|
v
Distributed State Store
/ | \
v v v
Shard0 Shard1 ShardN
|
atomic token/counter op
|
ALLOW / DENY
|
```

```
v
Application
```

39. Common Interview Follow-Ups

Which algorithm would you choose? Token bucket when controlled bursts plus sustained rate are required; explain alternatives/trade-offs.

Why not local counters only? Load balancing across servers would multiply the effective global quota.

How ensure atomicity? Server-side atomic operation rather than remote read-then-write.

What if the store is down? Explicit fail-open/fail-closed or conservative local fallback by endpoint risk.

How reduce remote calls? Local token leasing/caching at the cost of some accuracy.

What about hot keys? Hierarchical limits, dedicated handling, leases or approximate counters.

How expire idle users? TTL state after a safe inactivity/refill period.

Why steady_clock in C++? Rate refill depends on elapsed duration, not wall-clock jumps.

40. Interview-Ready Spoken Answer

"I'd first clarify whether the limit must be globally strict and whether bursts are allowed. For a typical API I'd choose a token bucket because it enforces a sustained refill rate while allowing a controlled burst. The limiter should sit at the API gateway so rejected requests do not consume downstream resources. The rate-limit key can be user, API key, IP or user plus endpoint. In a single process, I'd shard bucket state across maps and mutexes and use a monotonic clock for refill calculations. In a distributed gateway fleet, local counters alone are incorrect for a global quota, so I'd keep shared state in a partitioned low-latency store and perform refill/check/decrement atomically on the server side. For very high throughput, I'd consider leasing small token blocks to gateways to reduce remote operations, accepting a bounded accuracy trade-off. I'd explicitly define fail-open, fail-closed or conservative local fallback behavior if the shared store is unavailable. Finally I'd monitor allow/deny rates, decision latency, hot keys, store latency and fallback usage, and expire inactive bucket state with TTL."

41. Five Decisions to Remember

1. Need sustained rate + controlled burst -> TOKEN BUCKET.
2. Many app servers -> LOCAL COUNTERS ALONE DO NOT ENFORCE A GLOBAL LIMIT.
3. Concurrent updates -> CHECK + REFILL + CONSUME MUST BE ATOMIC.
4. Remote state too expensive -> TOKEN LEASING / LOCAL APPROXIMATION WITH EXPLICIT TRADE-OFF.
5. Limiter dependency fails -> DEFINE FAIL-OPEN / FAIL-CLOSED / LOCAL FALLBACK BEFOREHAND.

42. Final Revision Flow

```
Requirements
-> key + policy
-> algorithm comparison
-> token bucket
-> local thread safety
-> distributed shared state
-> atomic operation
-> partitioning/hot keys
-> token leasing
-> configuration/TTL
-> failure policy
-> retries/jitter
-> observability
-> C++ steady_clock/sharded locking
```